

Avance del Proyecto Integrador

Corte acumulativo — I Parcial · Semana 4

InventiGest

Sistema Web de Gestión de Inventario y Ventas

Autor	<u>Ragami Perez</u>
Asignatura	Arquitectura de Software — Proyecto Integrador
Fecha	29 de agosto de 2026

1. Información general del proyecto

Elemento	Descripción
Nombre	InventiGest — Sistema Web de Gestión de Inventario y Ventas.
Problema	Las PyMEs gestionan inventario y ventas de forma manual (hojas de cálculo o papel), lo que causa descuadres de stock, ventas perdidas por falta de disponibilidad actualizada y dificultad para generar reportes confiables.
Usuarios principales	Administrador (gestiona usuarios y consulta reportes), Vendedor (registra ventas) y Encargado de Inventario (mantiene productos y existencias).
Objetivo general	Centralizar en una aplicación web el control de inventario y ventas, con datos únicos, consistentes y consultables en tiempo real, y acceso controlado por roles.
Alcance inicial	Gestión de productos y stock, registro de ventas con descuento de existencias, control de acceso por roles, alertas de stock bajo y reportes básicos. No incluye app móvil ni facturación electrónica en esta etapa.

2. Requisitos del sistema

Requisitos funcionales (RF)

Código	Requisito funcional
RF01	El sistema permitirá iniciar sesión mediante credenciales (autenticación).
RF02	El sistema permitirá al Administrador registrar, editar y desactivar usuarios y asignarles un rol.
RF03	El sistema permitirá registrar y actualizar productos y sus categorías.
RF04	El sistema permitirá consultar el catálogo y la disponibilidad de stock.
RF05	El sistema permitirá registrar una venta, descontando el stock de forma automática.
RF06	El sistema permitirá generar un comprobante de la venta realizada.
RF07	El sistema permitirá registrar entradas o ajustes de inventario (reposición).
RF08	El sistema emitirá alertas cuando un producto alcance su stock mínimo.
RF09	El sistema permitirá generar reportes de ventas e inventario por periodo.
RF10	El sistema restringirá las funciones disponibles según el rol del usuario.

Requisitos no funcionales (RNF)

Código	Atributo	Requisito no funcional
RNF01	Seguridad	Las contraseñas se almacenarán con hash (bcrypt); el acceso será por roles (RBAC) y la comunicación sobre HTTPS.
RNF02	Rendimiento	Las consultas de catálogo responderán en menos de 2 segundos con hasta 1000 productos.
RNF03	Disponibilidad	El sistema estará operativo durante el horario comercial y contará con respaldos diarios de la base de datos.
RNF04	Usabilidad	Un vendedor podrá registrar una venta en 3 pasos o menos, con una interfaz clara.
RNF05	Mantenibilidad	El sistema se organizará en capas, con cada módulo de responsabilidad única.
RNF06	Escalabilidad	La API será sin estado (stateless) para soportar múltiples usuarios concurrentes.

Restricciones

- **Tecnología:** React + Node.js/Express + PostgreSQL (herramientas conocidas por el equipo).
- **Tiempo:** el desarrollo debe completarse dentro del semestre académico.
- **Recursos:** sin presupuesto para servicios de pago; se usarán planes gratuitos o infraestructura educativa.
- **Plataforma:** navegador web moderno; la app móvil queda fuera del alcance inicial.
- **Integraciones:** servicio de correo para notificaciones; facturación electrónica considerada opcional.

3. Actores y casos de uso principales

Actores

Actor	Responsabilidad / forma de interacción
Administrador	Gestiona usuarios y roles; consulta reportes globales del negocio.
Vendedor	Registra ventas y consulta la disponibilidad de productos.
Encargado de Inventario	Mantiene el catálogo y las existencias; atiende alertas de stock.

Casos de uso principales

Código	Caso de uso	Actor
CU-01	Iniciar sesión	Todos los roles
CU-02	Gestionar usuarios y roles	Administrador
CU-03	Registrar / actualizar productos y stock	Encargado de Inventario
CU-04	Consultar catálogo y disponibilidad	Vendedor
CU-05	Registrar una venta	Vendedor
CU-06	Recibir alertas de stock bajo	Encargado de Inventario
CU-07	Consultar reportes de ventas e inventario	Administrador

4. Diagrama de contexto

Muestra los límites del sistema: quién lo usa y con qué elementos externos se comunica.



Actores (azul) usan el sistema mediante el navegador. **Sistemas externos / terceros** (naranja) se comunican con InveniGest para correo, pagos, facturación y almacenamiento de imágenes.

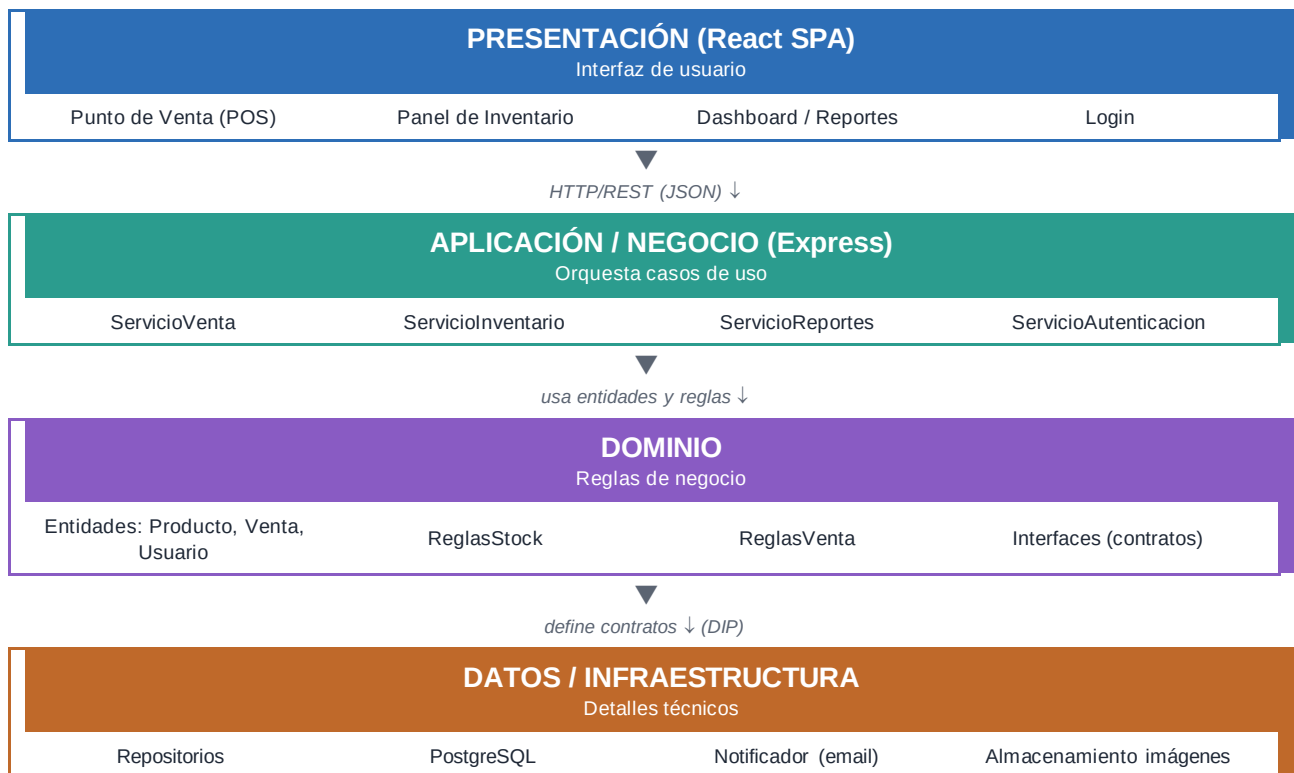
5. Atributos de calidad priorizados

Atributo	Necesidad	Impacto arquitectónico
Integridad de datos	Una venta y su descuento de stock deben registrarse juntos o no registrarse.	Base de datos relacional con operaciones transaccionales (ACID) y restricciones de integridad.
Seguridad	Roles con distintos permisos y datos sensibles de ventas y usuarios.	Autenticación con tokens (JWT), control de acceso por roles (RBAC) y HTTPS.
Mantenibilidad	La solución evolucionará durante el semestre.	Arquitectura en capas y separación de responsabilidades; comunicación por interfaces.

Estos tres atributos guían las decisiones técnicas y establecen la trazabilidad: necesidad del negocio → atributo → componente → decisión arquitectónica.

6. Vista lógica preliminar

Responde: ¿cómo está organizado internamente el sistema? Arquitectura en capas con backend modular.



7. Vista física / de despliegue preliminar

Responde: ¿dónde se ejecutará cada parte del sistema? Propuesta inicial de despliegue en la nube.

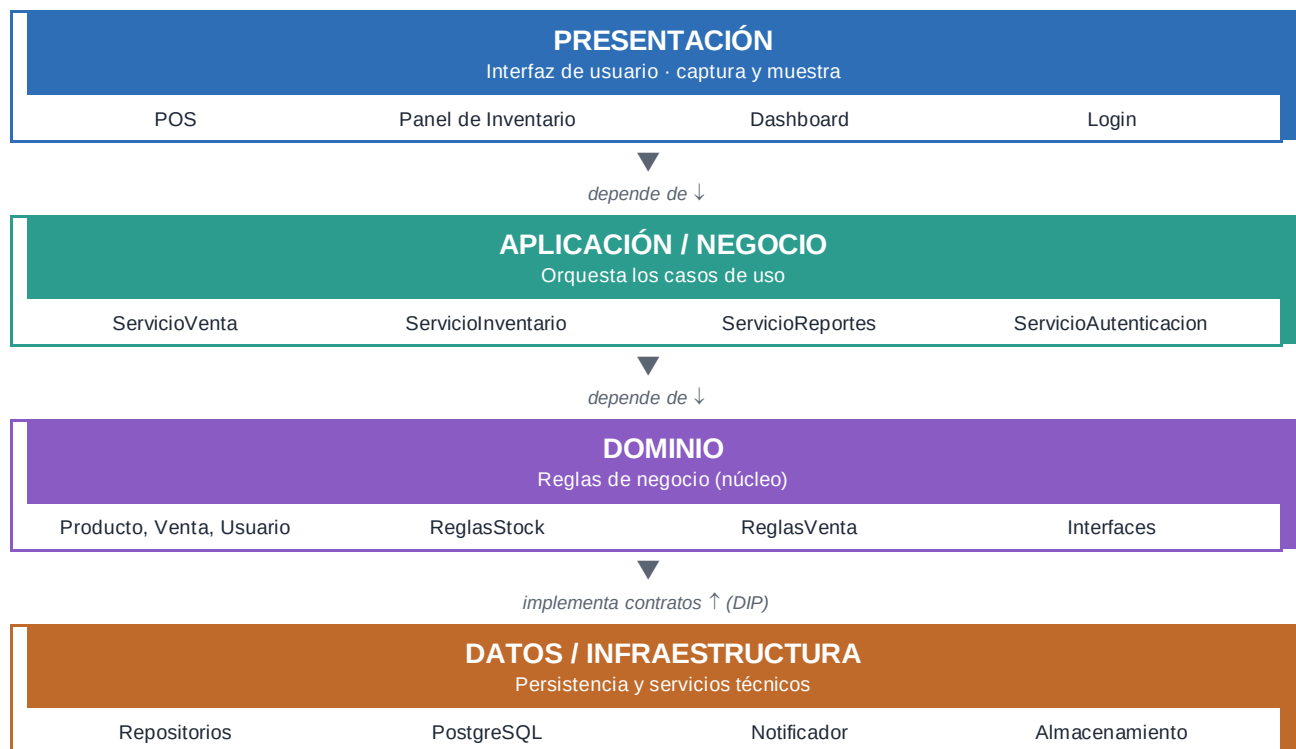
```

[ Usuario / Navegador ]
  | Internet (HTTPS)
  v
[ Servidor Web / CDN ] --- sirve el Frontend (React SPA)
  | REST / JSON
  v
[ Servidor de Aplicaciones (API Express) ] ----> [ Servicios externos ]
  |                                           - Correo / notificaciones
  |                                           - Pasarela de pagos (opc.)
  |                                           - Almacenamiento en nube
  v
[ Servidor de Base de Datos (PostgreSQL + respaldos) ]

```

8. Arquitectura en capas

La solución se organiza en cuatro capas; la dependencia fluye de arriba hacia abajo y las reglas del negocio se concentran en el Dominio (ver también la vista lógica de la sección 6).



9. Responsabilidades por capa

Capa	Responsabilidad / operaciones	Componentes	NO le corresponde
Presentación	Capturar datos, mostrar resultados y gestionar la interacción.	POS, Panel Inventario, Dashboard, Login	Reglas de negocio; consultas SQL
Aplicación	Orquestrar casos de uso; coordinar Dominio e Infraestructura.	ServicioVenta, ServicioInventario, ServicioReportes	Reglas del negocio o SQL concreto
Dominio	Definir y validar reglas (stock, totales, políticas).	Entidades, ReglasStock, ReglasVenta, Interfaces	Conocer pantallas, frameworks o SQL
Infraestructura	Persistir/recuperar datos; enviar correos; implementar contratos.	Repositorios, PostgreSQL, Notificador	Reglas de negocio o lógica de UI

10. Dependencias entre capas

Cada capa depende solo de la inmediatamente inferior; nadie depende de Presentación. El Dominio no depende de detalles técnicos: define interfaces que la Infraestructura implementa (DIP), de modo que las reglas del negocio no dependan de pantallas, bases de datos ni frameworks.

Capa	Puede depender de	Debe evitar depender de
Presentación	Aplicación (API REST)	Dominio o Infraestructura directamente
Aplicación	Dominio y sus interfaces	Detalles de PostgreSQL o de la UI
Dominio	Nada externo (solo de sí mismo)	React, Express, SQL, frameworks
Infraestructura	Dominio (implementa sus interfaces)	Presentación

Reducción del acoplamiento — contratos principales

Contrato	Responsabilidad	Usa → Implementa
IProductoRepository	Consultar, crear y actualizar productos y existencias.	Aplicación → Infraestructura
IVentaRepository	Registrar la venta y su detalle de forma transaccional.	Aplicación → Infraestructura
INotificacionService	Enviar alertas de stock bajo y confirmaciones.	Aplicación → Infraestructura

11. Decisiones arquitectónicas iniciales

Decisión 1 — Arquitectura en capas	
Problema	El sistema requiere separar la interfaz de las reglas del negocio y controlar las dependencias.
Decisión	Adoptar una arquitectura en capas (Presentación, Aplicación, Dominio, Infraestructura).
Justificación	Permite separar responsabilidades, aislar las reglas del negocio y controlar el acoplamiento.
Beneficio esperado	Mejora la mantenibilidad y la capacidad de evolución del sistema.
Riesgo / consecuencia	Incrementa inicialmente el número de componentes y clases.

Decisión 2 — Base de datos relacional transaccional (PostgreSQL)	
Problema	Una venta debe guardarse y descontar stock a la vez; el inventario no puede quedar inconsistente.
Decisión	Usar PostgreSQL y registrar la venta dentro de una transacción atómica (ACID).
Justificación	Existen relaciones claras (producto–venta–detalle–usuario) y se requieren operaciones transaccionales.
Beneficio esperado	Mayor consistencia e integridad de los datos de inventario y ventas.
Riesgo / consecuencia	Requiere administrar relaciones y migraciones; escalado horizontal más costoso que en NoSQL.

Decisión 3 — API REST desacoplada del frontend	
Problema	El frontend y la lógica evolucionan a distinto ritmo y a futuro podría añadirse una app móvil.
Decisión	Separar en una API REST (Express) independiente y una SPA (React) que la consume.
Justificación	El desacoplamiento permite desplegar/escalar cada parte por separado y reutilizar la API.
Beneficio esperado	Flexibilidad, escalabilidad y reutilización de la misma lógica para web y móvil.
Riesgo / consecuencia	Mayor complejidad inicial: manejo de CORS, versionado de la API y autenticación por token.

12. Estructura inicial del repositorio

```

inventigest/
+-- frontend/          # Presentacion (React SPA)
+-- backend/
|   +-- src/presentation/ (rutas y controllers Express)
|   +-- src/application/  (services, dtos, interfaces)
|   +-- src/domain/       (entities, rules, interfaces)
|   \-- src/infrastructure/ (repositories, database, services)
+-- documentacion/      # este documento y diagramas
\-- README.md

```

Dato del repositorio	Valor
URL del repositorio Git	github.com/xddd-hub-star/InventiGest-Arquitectura
Rama principal	main

Conclusión

El avance evidencia coherencia entre el problema, los requisitos, los atributos de calidad y la arquitectura: cada necesidad del negocio se traduce en un requisito, un atributo de calidad y un componente o decisión concretos. La integridad de los datos motiva una base relacional transaccional; la mantenibilidad y la evolución motivan la arquitectura en capas con reglas aisladas en el Dominio; y la seguridad se refleja en autenticación y control por roles. La propuesta es realista con la tecnología y el tiempo disponibles, y podrá refinarse en las próximas semanas.

Trazabilidad: necesidad del negocio → requisito → atributo de calidad → componente → decisión arquitectónica.